



PYTHON NOTES — PYTHON DICTIONARIES AND TUPLES (CONTINUED)



TOPIC: TUPLES (PART 2)

15. THE `.count()` METHOD

The `.count()` method counts how many times a value appears in a tuple.

Example

```
numbers = (1, 2, 3, 2, 2, 5)

print(numbers.count(2))
```

Output

```
3
```

The number `2` appears three times.

Another example:

```
fruits = ('apple', 'banana', 'apple')

print(fruits.count('apple'))
```

Output:

```
2
```

Important

`.count()` does **not** change the tuple.

It only counts.

16. THE `.index()` METHOD

The `.index()` method returns the position (index) of the **first occurrence** of an item.

Example

```
fruits = (  
    'apple',  
    'banana',  
    'mango'  
)  
  
print(fruits.index('banana'))
```

Output:

1

Remember:

```
0 → apple  
1 → banana  
2 → mango
```

Important

If the item does not exist:

```
fruits.index('orange')
```

Python raises an error.

17. TUPLES VS LISTS

Although tuples and lists are very similar, they have important differences.

Lists	Tuples
Created with <code>[]</code>	Created with <code>()</code>

Lists	Tuples
Mutable	Immutable
Can add items	Cannot add items
Can remove items	Cannot remove items
Can update items	Cannot update items
Many methods	Only a few methods
Best for changing data	Best for fixed data

18. WHEN SHOULD I USE A TUPLE?

Use a tuple when the information should stay the same.

Examples:

```
days = (  
    'Monday',  
    'Tuesday',  
    'Wednesday',  
    'Thursday',  
    'Friday',  
    'Saturday',  
    'Sunday'  
)
```

The days of the week do not change.

Example:

```
months = (  
    'January',  
    'February',  
    ...  
)
```

Months remain fixed.

Example:

```
rgb = (  
    255,  
    255,  
    255  
)
```

RGB colour values represent one colour.

Example:

```
coordinate = (  
    40.7128,  
    -74.0060  
)
```

Coordinates are usually treated as one fixed pair.

19. REAL-WORLD USES OF TUPLES

Tuples appear in many real Python programs.

Examples include:

- GPS coordinates
- RGB colours
- Dates
- Months
- Days
- Mathematical points
- Fixed settings
- Configuration values
- Database records
- Dictionary values
- Returning multiple values from functions

Although tuples are simple, they are extremely useful.

20. OUR LIBRARY MANAGEMENT SYSTEM PROJECT

After completing Dictionaries, we built another project:

Library Management System

Instead of managing students, we managed books.

Each book stored:

- title
- author
- genres
- year

Example:

```
books = {  
    'book1': {  
        'title': 'Atomic Habits',  
        'author': 'James Clear',  
        'genres': (  
            'Self-help',  
            'Productivity'  
        ),  
        'year': 2018  
    }  
}
```

Notice:

```
'genres'
```

was stored as a **tuple**.

21. WHY DID WE STORE GENRES AS A TUPLE?

We decided that a book's genres should remain fixed.

For example:

Self-help
Productivity

are characteristics of the book.

The user can update:

- title
- author
- year

But **genres** remain unchanged.

To demonstrate this idea, we wrote:

```
if info_key == 'genres':  
    print(  
        'Genres cannot be updated because they are stored as a tuple.'  
    )
```

This was an intentional design decision.

It helped us understand one practical reason for choosing tuples instead of lists.

22. WHY WE DID NOT CREATE A STANDALONE TUPLES PROJECT

Unlike dictionaries, tuples have very few operations.

They cannot:

- add items
- remove items
- update items

Because of this, creating an entire project based only on tuples would not have been very useful.

Instead, we learned all of the important tuple concepts through:

- examples
- practice questions
- small exercises

Then we naturally combined tuples with dictionaries inside the **Library Management System**.

This better reflects how tuples are used in real Python programs.

In practice, tuples usually work **together** with other data structures rather than by themselves.

23. WHAT I CAN DO NOW

After learning Tuples, I can:

- ✓ Create tuples
 - ✓ Access tuple items
 - ✓ Use positive indexes
 - ✓ Use negative indexes
 - ✓ Slice tuples
 - ✓ Loop through tuples
 - ✓ Count items with `len()`
 - ✓ Pack tuples
 - ✓ Unpack tuples
 - ✓ Create single-item tuples
 - ✓ Use `.count()`
 - ✓ Use `.index()`
 - ✓ Understand tuple immutability
 - ✓ Know when to use tuples instead of lists
 - ✓ Combine tuples with dictionaries
-

MY PROGRESS

Dictionaries

- ✓ Creating dictionaries
 - ✓ Keys
 - ✓ Values
 - ✓ Accessing data
 - ✓ Adding data
 - ✓ Updating data
 - ✓ Removing data
 - ✓ `pop()`
 - ✓ `.keys()`
 - ✓ `.values()`
 - ✓ `.items()`
 - ✓ Nested dictionaries
 - ✓ Student Management System
-

Tuples

- ✓ Creating tuples
- ✓ Indexing
- ✓ Negative indexing
- ✓ Slicing
- ✓ Looping
- ✓ `len()`

- ✓ Single-item tuples
 - ✓ Packing
 - ✓ Unpacking
 - ✓ `.count()`
 - ✓ `.index()`
 - ✓ Tuple vs List
 - ✓ Immutability
 - ✓ Library Management System
-

DICTIONARIES AND TUPLES COMPLETE

By completing these topics, I now understand two of Python's most important data structures.

I understand when to use:

- Lists
- Dictionaries
- Tuples

I can now organize information in different ways depending on the problem I am solving.

I have also completed two menu-driven projects:

- ✓ Student Management System
- ✓ Library Management System

These projects combined many Python concepts, including:

- Functions
- Dictionaries
- Tuples
- Loops
- Conditions
- User input
- Nested dictionaries
- Updating data

- Adding data
- Removing data
- Menu-driven programs

They helped me gain practical experience instead of only learning theory.



NEXT TOPIC

The next topic in my Python journey is:



FILE HANDLING

In the next chapter, I will learn how to:

- Open files
 - Read files
 - Write files
 - Append to files
 - Work with `.txt` files
 - Save information permanently
 - Build programs that remember data even after they close
-



DICTIONARIES AND TUPLES COMPLETE

I now understand how dictionaries organize information using **key-value pairs** and how tuples safely store **fixed data**.

These two data structures are important foundations for building larger Python programs, and I will continue using them throughout the rest of my Python learning journey.



DICTIONARIES & TUPLES →  COMPLETE

Next: FILE HANDLING 